MySkill | #RintisKarirImpian

Portofolio

Intensive Bootcamp Data Analysis

Name: Siti Asih Rahmahaya

Outline Tasks

1. Basic Statistics
2. Data formatting dan Validation
3. SQL Basic
4. SQL Basic : Analyzing Business Data
5. SQL For Data Analysis 1
6. SQL For Data Analysis 2
7. SQL Technical Interview Case Studies
8. Data Analysis with Python 1: Basic Python + Project 1
9. Data Analysis with Python 2: Work with Numpy and Pandas
10. Data Analysis with Python 3: Case Study
11. Data Visualisation 1: Basic Data Viz + Project
12. Data Visualisation 2: Google Data Studio

MySkill | #RintisKarirImpian

Portofolio - Intensive Bootcamp

Basic Statistics

Link Excel Exercise: task 3 statistics

Standar Deviation

Date	Visitor of the Public Service
2022-12-01	4
2022-12-02	2
2022-12-03	5
2022-12-04	10
2022-12-05	7
2022-12-06	8
2022-12-07	10
2022-12-08	5
2022-12-09	8
2022-12-10	4
2022-12-11	9
2022-12-12	6
2022-12-13	10
2022-12-14	8
2022-12-15	4
2022-12-16	6
2022-12-17	8
2022-12-18	5
2022-12-19	7
2022-12-20	3
2022-12-21	9
2022-12-22	7
2022-12-23	5
2022-12-24	4
2022-12-25	7
2022-12-26	7
2022-12-27	7
2022-12-28	5
2022-12-29	8
2022-12-30	5

Case:
If you are the leader of one institution in Surabaya, how many chair that you need to prepared to cover 68% of all visitior at least will get the seat?

Answer:

Mean (μ) =	6.43
Std (σ) =	2.14
with target coverage 68%	
Assume normal distribution	

Since the problem asks for the minimum number of chairs so that **at least 68% of visitors can get a seat**, this can be interpreted as finding the **68th percentile** of the normal distribution.

That means:	$P(X \leq x) = 0.68$
From the normal distribution table:	$z \approx 0.47$

z	.00	.01	.02	.03	.04	.05	.06	.07	.08	.09
0.0	.5000	.5040	.5080	.5120	.5160	.5199	.5239	.5279	.5319	.5359
0.1	.5398	.5438	.5478	.5517	.5557	.5596	.5636	.5675	.5714	.5753
0.2	.5793	.5832	.5871	.5910	.5948	.5987	.6026	.6064	.6103	.6141
0.3	.6179	.6217	.6255	.6293	.6331	.6368	.6406	.6443	.6480	.6517
0.4	.6554	.6591	.6628	.6664	.6700	.6736	.6772	.6808	.6844	.6879
0.5	.6915	.6950	.6985	.7019	.7054	.7088	.7123	.7157	.7190	.7224
0.6	.7257	.7291	.7324	.7357	.7389	.7422	.7454	.7486	.7517	.7549
0.7	.7580	.7611	.7642	.7673	.7704	.7734	.7764	.7794	.7823	.7852
0.8	.7881	.7910	.7939	.7967	.7995	.8023	.8051	.8078	.8106	.8133
0.9	.8159	.8186	.8212	.8238	.8264	.8289	.8315	.8340	.8365	.8389

Using:	$x = \mu + z\sigma$		
x =	6.43 + (0.47)(2.14)		
x =	7.441245081	$\lceil x \rceil \approx$	8 chairs should be prepared.

Note: If using the empirical rule (±1σ covers ~68% around the mean), the upper bound would be 8.57 (~9 chairs). However, for capacity planning, using the 68th percentile (8 chairs) is more appropriate because it directly represents the minimum number of seats needed to accommodate 68% of daily visitors.

MySkill

Z-Score

Which option should Udin choose when considering the cost of living based on the environment?

Udin's Friends' Data	Monthly Cost of Living	
	Bekasi	Tuban
1	4,800,000	2,500,000
2	5,000,000	2,100,000
3	4,200,000	3,000,000
4	4,600,000	3,000,000
5	4,500,000	2,500,000
6	5,200,000	2,400,000

	Bekasi	Tuban
Job Offers with Salary To Udin (x)	5,000,000	3,500,000

Mean Sample (μ)	4,716,667	2,583,333
Std. Sample (σ)	360092.5807	354494.9459
Z Scores	0.79	2.59
z = (x-μ)/σ	79%	99.52%

ps:
A high Z-score means the salary (x) exceeds the average cost of living (μ), indicating greater financial security.

If the z-score is negative, for example:

$$z = -1$$

means:
Salary below the average cost of living lead to increased insecurity.

z score Bekasi

z	.00	.01	.02	.03	.04	.05	.06	.07	.08	.09
0.0	.5000	.5040	.5080	.5120	.5160	.5199	.5239	.5279	.5319	.5359
0.1	.5398	.5438	.5478	.5517	.5557	.5596	.5636	.5675	.5714	.5753
0.2	.5793	.5832	.5871	.5910	.5948	.5987	.6026	.6064	.6103	.6141
0.3	.6179	.6217	.6255	.6293	.6331	.6368	.6406	.6443	.6480	.6517
0.4	.6554	.6591	.6628	.6664	.6700	.6736	.6772	.6808	.6844	.6879
0.5	.6915	.6950	.6985	.7019	.7054	.7088	.7123	.7157	.7190	.7224
0.6	.7257	.7291	.7324	.7357	.7389	.7422	.7454	.7486	.7517	.7549
0.7	.7580	.7611	.7642	.7673	.7704	.7734	.7764	.7794	.7823	.7852
0.8	.7881	.7910	.7939	.7967	.7995	.8023	.8051	.8078	.8106	.8133
0.9	.8159	.8186	.8212	.8238	.8264	.8289	.8315	.8340	.8365	.8389
1.0	.8413	.8438	.8461	.8485	.8508	.8531	.8554	.8577	.8599	.8621

z score Tuban

1.8	.9641	.9649	.9656	.9664	.9671	.9678	.9686	.9693	.9699	.9706
1.9	.9713	.9719	.9726	.9732	.9738	.9744	.9750	.9756	.9761	.9767
2.0	.9772	.9778	.9783	.9788	.9793	.9798	.9803	.9808	.9812	.9817
2.1	.9821	.9826	.9830	.9834	.9838	.9842	.9846	.9850	.9854	.9857
2.2	.9861	.9864	.9868	.9871	.9875	.9878	.9881	.9884	.9887	.9890
2.3	.9893	.9896	.9898	.9901	.9904	.9906	.9909	.9911	.9913	.9916
2.4	.9918	.9920	.9922	.9925	.9927	.9929	.9931	.9932	.9934	.9936
2.5	.9938	.9940	.9941	.9943	.9945	.9946	.9948	.9949	.9951	.9952
2.6	.9953	.9955	.9956	.9957	.9959	.9960	.9961	.9962	.9963	.9964

This means that the salary in Bekasi is only 0.79σ above the average cost of living, whereas the salary in Tuban is 2.59σ above the average cost of living.

Conclusion:
Udin would be better off choosing Tuban; even though the nominal salary is lower, the offer is far more advantageous when considered relative to the local cost of living.

MySkill

Percentile

Before Sorted		After Sorted
Transaction ID	service duration	service duration
1	1	1
2	4	1
3	8	1
4	5	1
5	5	1
6	1	2
7	10	2
8	5	2
9	6	3
10	1	3
11	1	3
12	10	4
13	2	4
14	1	4
15	4	4
16	6	4
17	8	5
18	4	5
19	9	5
20	5	5
21	10	5
22	7	5
23	10	6
24	2	6
25	10	6
26	2	6
27	7	7
28	10	7
29	7	7
30	4	7
31	3	7
32	10	8

In one money transfer company, the expected of Percentile 90 SLA is under 5 mins to ensure the customer satisfaction of the service provided

- Is the company was achieved P90 Satisfaction Level Condition?
- What is your recommendation to product team to solve this condition?

The client will not make the purchase because the actual time is 10 minutes, whereas the promised P90 is 5 minutes (meaning the promise is that 90% of transactions will take 5 minutes or less).

Answer:

(Method 1) Direct Percentile

Because the dataset contains actual transaction durations, the most appropriate way is to calculate the **90th percentile directly**.

Step:
Sort all 50 transactions from smallest to largest.

P90 position: $P90 = 0.9 \times 50 = 45$

So we look at the **45th observation**.

45th observation the value is 10 mins

Thus: P90 = 10 mins
Target SLA = P90 ≤ 5
Since:

$$10 > 5$$

The company **did not achieve** the P90 SLA.

(Method 2) Normal approximation (z-score)

Assuming the data is normally distributed:

mean:	6.14		
std:	3.05734312		
target SLA (x):	5		
z-score:	-0.3728728	z ≈ 0.3557	≈ 35.58%

Since: 36% < 90% The company did not achieve the SLA.

I recommend the product team:
1. Analyze bottlenecks in transaction flow.
2. Identify slow transaction segments (large amount, interbank, peak hours).
3. Optimize backend processing to reduce latency.
4. Monitor P90 and P95 regularly instead of relying only on average SLA.

MySkill

MySkill | #RintisKarirImpian

Portofolio - Intensive Bootcamp

Data Formatting and Validation

Link Excel Exercise: task 5 superstore

Data Formatting

1. Use **=IMPORTRANGE** function if necessary to retrieve data from another worksheet.

2. Format the discount column as a percentage.

3. Format the sales and profit columns as currency in USD ($) with a maximum of two decimal places.

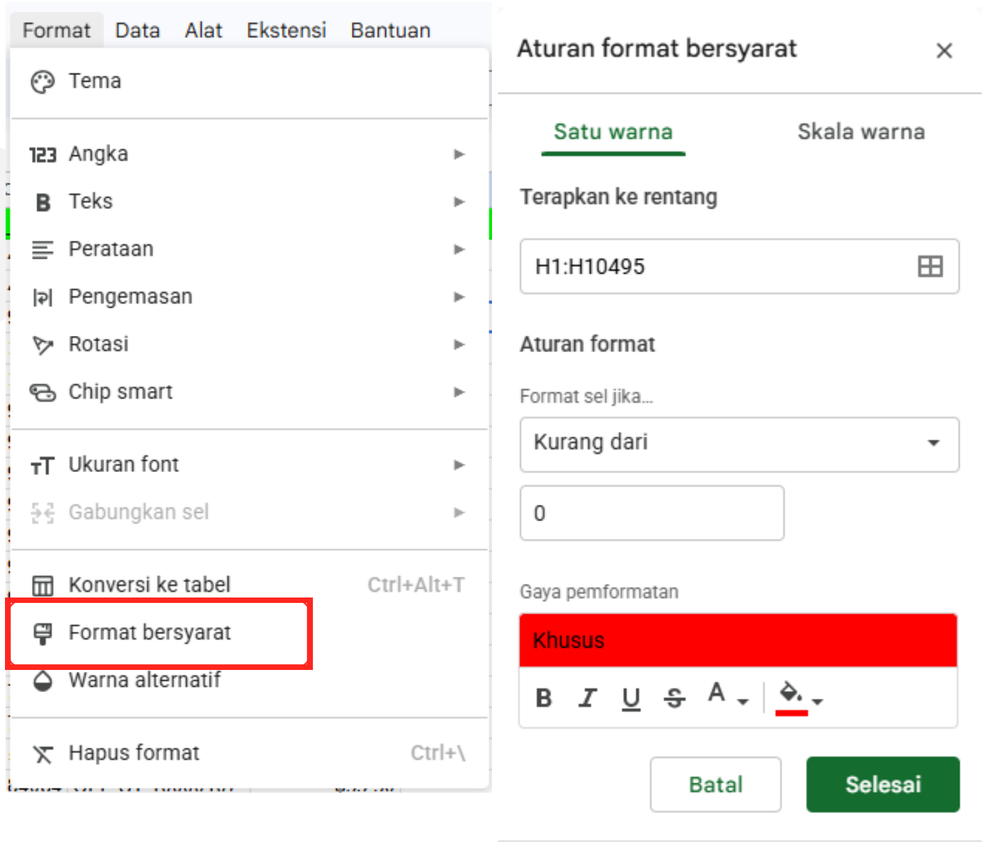
discount
0%
0%
0%
45%
20%
0%
0%
20%
20%
0%
20%
20%
20%
20%

E
sales
$261.96
$731.94
$14.62
$957.58
$22.37
$48.86
$7.28
$907.15
$18.50
$114.90
$1,706.18
$911.42
$15.55
$407.98
$68.81
$2.54
$665.88
$55.50
$8.56
$213.48
$22.72
$19.46
$60.34
$71.37

H
profit
$41.91
$219.58
$6.87
-$383.03
$2.52
$14.17
$1.97
$90.72
$5.78
$34.47
$85.31
$68.36
$5.44
$132.59
-$123.86
-$3.82
$13.32
$9.99
$2.48
$16.01
$7.38
$5.06
$15.69

Conditional Formatting

1. In the “Profit” column, mark negative profits in red.



H
profit
$41.91
$219.58
$6.87
-$383.03
$2.52
$14.17
$1.97
$90.72
$5.78
$34.47
$85.31
$68.36
$5.44
$132.59
-$123.86
-$3.82
$13.32
$9.99
$2.48
$16.01
$7.38
$5.06
$15.69
-$1.02
$240.26

2. Change the data in the Ship_Mode column to Proper Case for the Region field, using the function **=PROPER(text).**

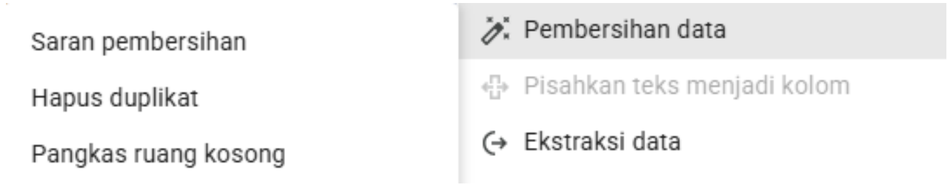
N	O	P	Q	R	S	T
Ship_Mode	Customer_Name	Segment	Country	City	State	Region
Second Class	Claire Gute	Consumer	United States	Henderson	Kentucky	South
Second Class	Claire Gute	Consumer	United States	Henderson	Kentucky	South
Second Class	Darrin Van Huff	Corporate	United States	Los Angeles	California	West
Standard Class	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Florida	South
Standard Class	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Florida	South
Standard Class	Brosina Hoffman	Consumer	United States	Los Angeles	California	West
Standard Class	Brosina Hoffman	Consumer	United States	Los Angeles	California	West
Standard Class	Brosina Hoffman	Consumer	United States	Los Angeles	California	West
Standard Class	Brosina Hoffman	Consumer	United States	Los Angeles	California	West
Standard Class	Brosina Hoffman	Consumer	United States	Los Angeles	California	West
Standard Class	Brosina Hoffman	Consumer	United States	Los Angeles	California	West
Standard Class	Brosina Hoffman	Consumer	United States	Los Angeles	California	West
Standard Class	Andrew Allen	Consumer	United States	Concord	North Carolina	South
Standard Class	Irene Maddox	Consumer	United States	Seattle	Washington	West
Standard Class	Harold Pawlan	Home Office	United States	Fort Worth	Texas	Central
Standard Class	Harold Pawlan	Home Office	United States	Fort Worth	Texas	Central
Standard Class	Pete Kriz	Consumer	United States	Madison	Wisconsin	Central
Second Class	Alejandro Grove	Consumer	United States	West Jordan	Utah	West
Second Class	Zuschuss Donat	Consumer	United States	San Francisco	California	West
Second Class	Zuschuss Donat	Consumer	United States	San Francisco	California	West
Second Class	Zuschuss Donat	Consumer	United States	San Francisco	California	West
Standard Class	Ken Black	Corporate	United States	Fremont	Nebraska	Central
Standard Class	Ken Black	Corporate	United States	Fremont	Nebraska	Central
Second Class	Sandra Flanagai	Consumer	United States	Philadelphia	Pennsylvania	East
Standard Class	Emily Burns	Consumer	United States	Orem	Utah	West
Second Class	Eric Hoffmann	Consumer	United States	Los Angeles	California	West
Second Class	Eric Hoffmann	Consumer	United States	Los Angeles	California	West

MySkill

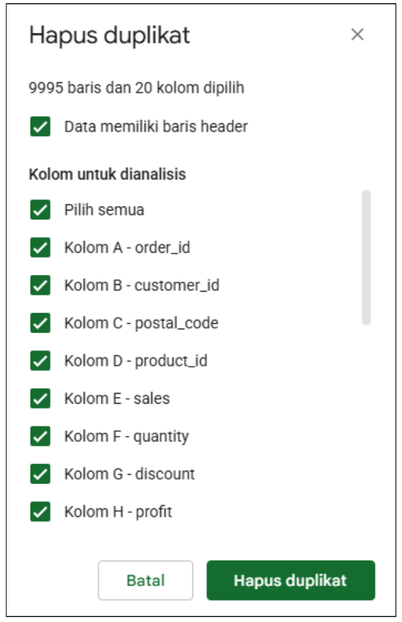
Data Duplicate Cleaning

Check whether the supermarket data contains any duplicate transactions.

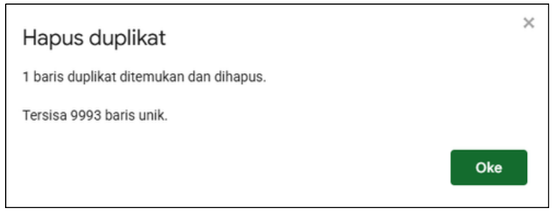
1. Select all the data on the sheet, then use the duplicate cleanup feature in the data tools.



2. Select all data to check for unique entries and remove duplicates.

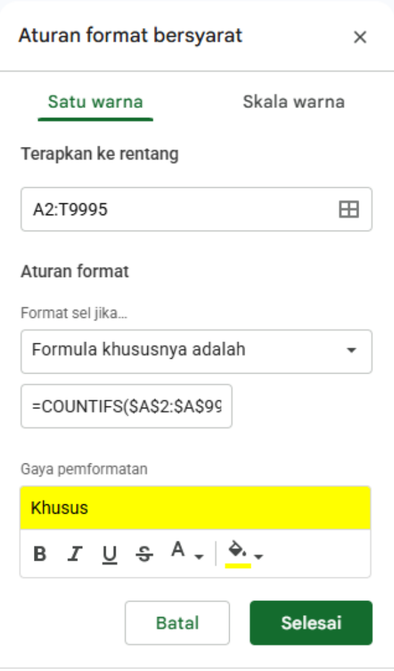


3. 1 duplicate row found to be deleted

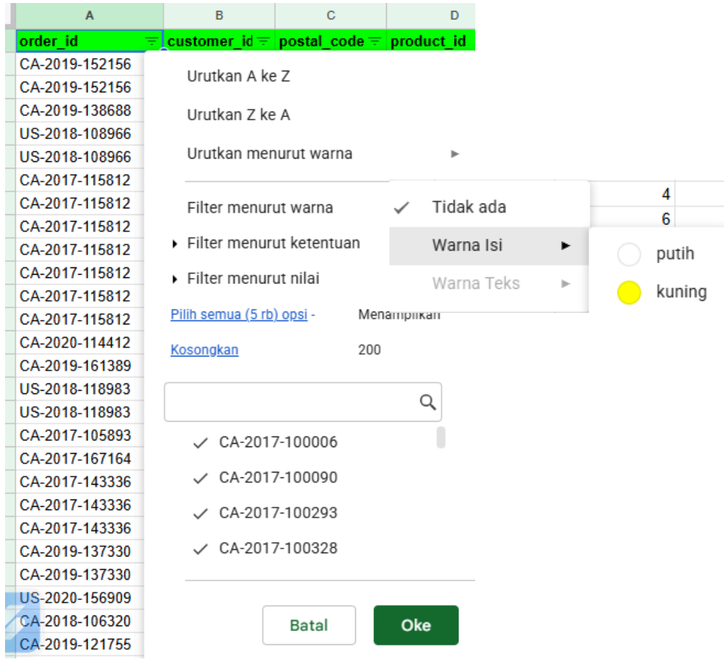


4. To determine which rows were deleted, conditional formatting is used for all data. Duplicate rows are marked with a yellow background. The function used for this is:

=COUNTIFS($A$2:$A$9995, $A2, $D$2:$D$9995, $D2, $E$2:$E$9995, $E2, $F$2:$F$9995, $F2, $H$2:$H$9995, $H2)>1



5. Filter the data in one of the columns, for example, `order_id` by the color yellow to view the duplicate rows.



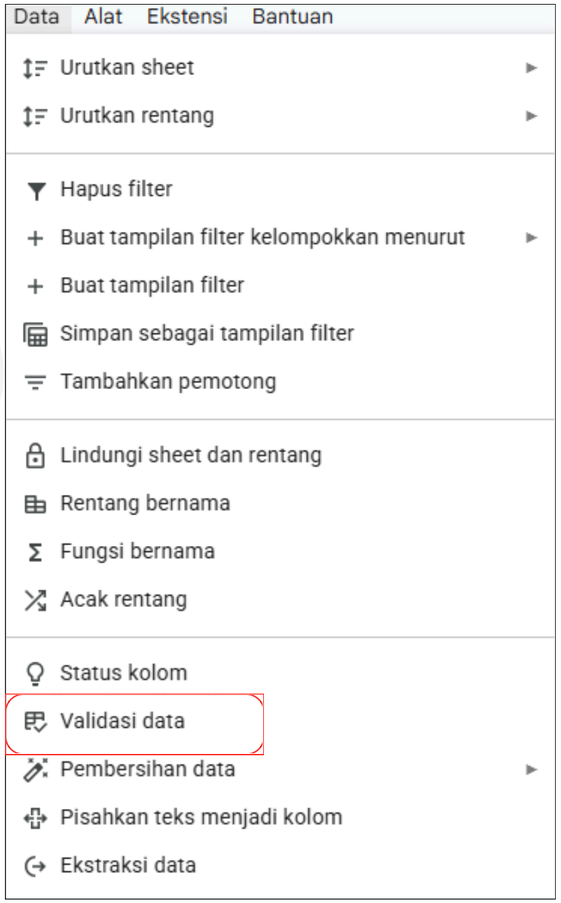
6. After that, the duplicate rows were identified: rows 3407 and 3408.

	A	B	C	D	E	F	G
1	order_id	customer_id	postal_code	product_id	sales	quantity	discount
3407	US-2017-150119	LB-16795	43229	FUR-CH-10002965	$281.37	2	30%
3408	US-2017-150119	LB-16795	43229	FUR-CH-10002965	$281.37	2	30%

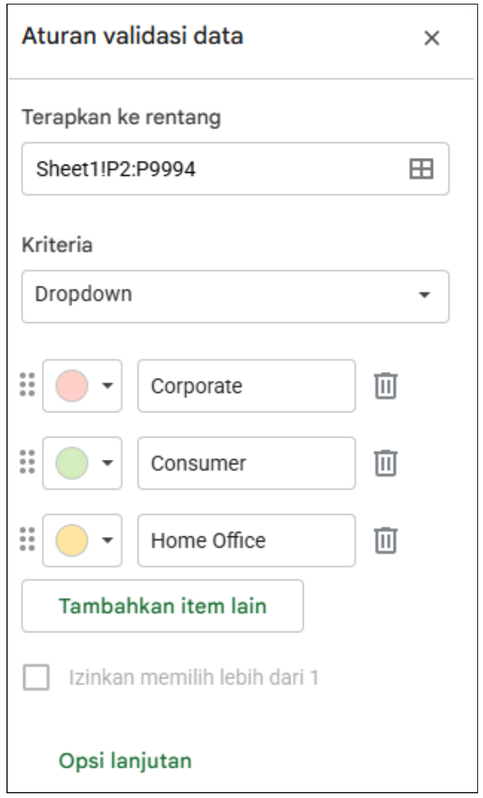
Data Validation

For the Segment section, apply data validation so that the entered data is one of the following options: Corporate, Consumer, or Home Office.

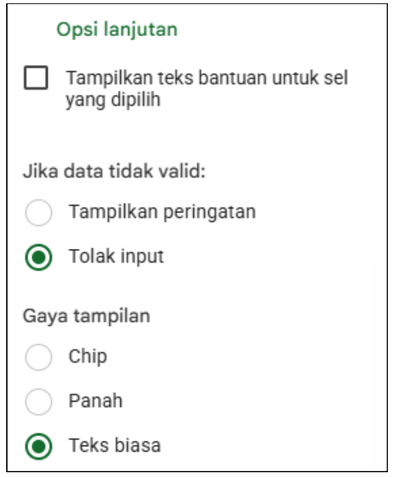
1. Use the Data Validation tools.



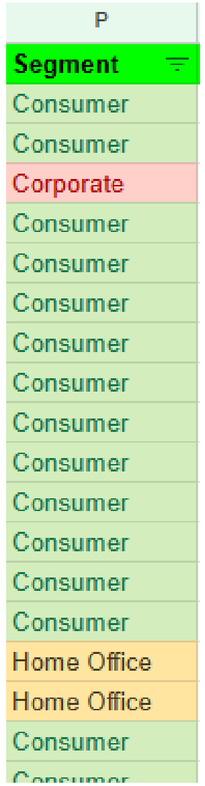
2. In the data validation section, set the dropdown item criteria to only the words "Corporate," "Consume," and "Home Office."



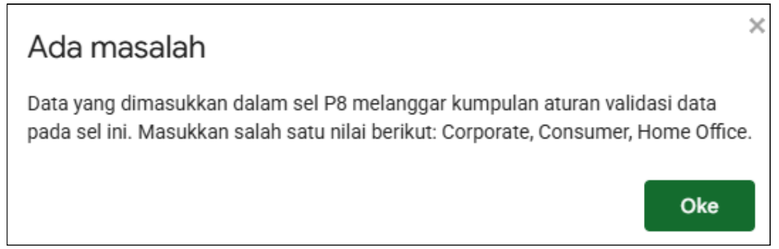
3. With advanced options for other inputs, the data is invalid (rejected).



4. Once validated, the column is colored differently according to the input parameters.



5. If there is other input that is invalid then there will be a rejection pop-up



MySkill

MySkill | #RintisKarirImpian

Portofolio - Intensive Bootcamp

SQL Basic

Overview:
Basic SQL query implementations using SELECT, WHERE, DISTINCT, filtering logic (AND, IN), and date range conditions (BETWEEN).

SQL Basic

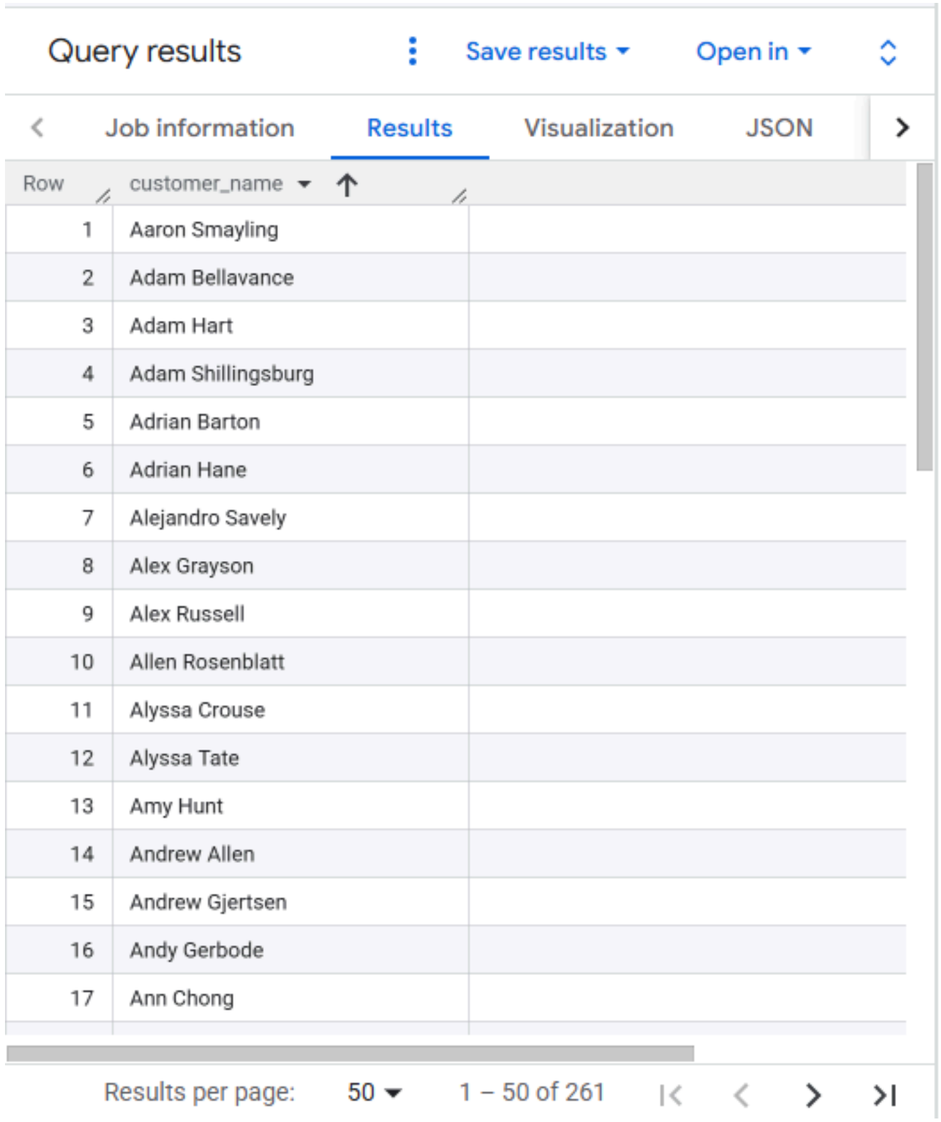
Key SQL Concepts Used:

- **SELECT DISTINCT:** Retrieves unique values from the column to eliminate duplicate customer entries in the output.
- **WHERE** Filtering: Used to apply conditional criteria for querying targeted data records.
- **AND** Operator: Combines multiple criteria where all conditions must be true simultaneously (e.g., Category & Sub-Category).
- **IN (...)** Clause: Evaluates multiple potential values within a single column, used to filter multiple segments ('Corporate', 'Home Office').
- **BETWEEN** ... **AND** ...: Filters records within an inclusive range, used for extracting date ranges ('2018-01-01' to '2018-12-31').

Question 1 Display the names of customers in the 'Consumer' segment who have purchased a table.

- SELECT DISTINCT customer_name: Fetches unique customer names to avoid duplicate rows.
- FROM tokopaedi.orders: Specifies the target table containing order data.
- WHERE category = "Furniture": Filters the dataset to only include products under the Furniture category.
- AND subcategory = "Tables": Narrow down the filter specifically to customers who bought Tables.

```
select
 distinct
 customer_name,
from
 `tokopaedi.orders`
where
 category = "Furniture"
 AND
 subcategory = "Tables";
```



The query executed successfully and returned 261 rows. Each row displays a unique customer name from the 'Consumer' segment who purchased a Table

#RintisKarirImpian

Question 2

Display the names of customers in the 'Corporate' and 'Home Office' segments who are from Los Angeles and made transactions during 2018.

- SELECT DISTINCT customer_name: Selects unique customer names to prevent duplicates.
- FROM tokopaedi.orders: Points to the main orders table.
- WHERE segment IN ('Corporate', 'Home Office'): Filters for customers belonging to either the Corporate or Home Office segment.
- AND city = 'Los Angeles': Restricts the location specifically to customers based in Los Angeles.
- AND order_date BETWEEN '2018-01-01' AND '2018-12-31': Filters for transactions that occurred within the 2018 calendar year.

```sql
SELECT DISTINCT
  customer_name
FROM
  `tokopaedi.orders`
WHERE
  segment IN ('Corporate', 'Home Office')
  AND city = 'Los Angeles'
  AND order_date BETWEEN '2018-01-01' AND '2018-12-31';
```

Query results

Results | Visualization

Row	customer_name
1	Ann Blume
2	Mike Caudle
3	Linda Southworth
4	Grace Kelly
5	Anthony Jacobs
6	Jennifer Braxton
7	Rick Wilson
8	Alice McCarthy
9	Yoseph Carroll
10	Shaun Chance
11	Frank Atkinson
12	Roy Phan
13	Karen Bern
14	Eleni McCrary
15	Monica Federle
16	Bill Tyler

Results per page: 50

1 – 42 of 42

The query returned 42 rows of unique customer names matching all criteria. These are Corporate/Home Office customers in Los Angeles who bought items in 2018.

MySkill | #RintisKarirImpian

Portofolio - Intensive Bootcamp

SQL Basic : Analyzing Business Data

Overview:
Advanced business data analysis using SQL to identify loss-making and highly profitable transactions using range filters (BETWEEN), condition logic (profit < 0 / profit > 0), and dataset sorting (ORDER BY ASC/DESC).

Question 1

Query only the loss-making transactions that occurred in the city of Los Angeles between 2018 and 2019, sorted by the largest loss.

- SELECT ...: Specifies key order, customer, product, city, and profit columns to inspect.
- WHERE city = 'Los Angeles': Restricts the location strictly to Los Angeles.
- AND order_date BETWEEN '2018-01-01' AND '2019-12-31': Filters orders made between 2018 and 2019.
- AND profit < 0: Filters only transactions that resulted in a financial loss.
- ORDER BY profit ASC: Sorts results in ascending order to showcase the largest loss first.

```sql
SELECT
  order_id,
  customer_name,
  product_name,
  city,
  profit
FROM
  `tokopaedi.orders`
WHERE
  city = 'Los Angeles'
  AND order_date BETWEEN '2018-01-01' AND '2019-12-31'
  AND profit < 0
ORDER BY
  profit ASC;
```

This script will process 1.36 MB when run.

Query results

Chat | Save results | Open in

Job information | Results | Visualization | JSON | Execution details | Execution graph

Row	order_id	customer_name	product_name	city	profit
1	CA-2019-113243	Olvera Toch	Bretford "Just In Time" Height-Adjustable Multi-Task Work Tables	Los Angeles	-217.048
2	CA-2018-153388	Pauline Chand	Hon 2111 Invitation Series Corn...	Los Angeles	-175.8708
3	CA-2019-163594	Jason Fortune-	Global Geo Office Task Chair, Gr...	Los Angeles	-36.441
4	CA-2019-142398	Beth Paige	Hon Comfortask Task/Swivel C...	Los Angeles	-31.9144
5	CA-2019-114972	Phillip Flathmann	Global Deluxe High-Back Office ...	Los Angeles	-29.9178
6	CA-2018-102316	Dave Hallsten	Global Deluxe Steno Chair	Los Angeles	-20.7846
7	US-2019-126452	Scot Coram	Motorola HK250 Universal Bluet...	Los Angeles	-20.691
8	CA-2018-106257	Eugene Barchas	Iceberg OfficeWorks 42- Round ...	Los Angeles	-15.098
9	US-2019-139486	Logan Haushalter	Motorola HK250 Universal Bluet...	Los Angeles	-12.4146
10	US-2019-125969	Lindsay Shagiari	Global Value Mid-Back Manager...	Los Angeles	-9.1602
11	CA-2018-131597	Stefania Perrino	KI Conference Tables	Los Angeles	-8.5068
12	CA-2018-129546	Roy Phan	Motorola HK250 Universal Bluet...	Los Angeles	-8.2764
13	US-2019-126452	Scot Coram	Anker Astro Mini 3000mAh Ultr...	Los Angeles	-7.996
14	CA-2019-129868	Mike Caudle	Global Armless Task Chair, Roy...	Los Angeles	-5.4882
15	CA-2019-135265	Christopher Conant	Global Leather Task Chair, Black	Los Angeles	-3.5996

Results per page: 50 | 1 – 16 of 16

The query executed successfully and **returned 16 loss-making transactions**. The biggest financial loss in Los Angeles was recorded under Order ID CA-2019-113243 for a Bretford Work Table, incurring a net loss of **-$217.05.**

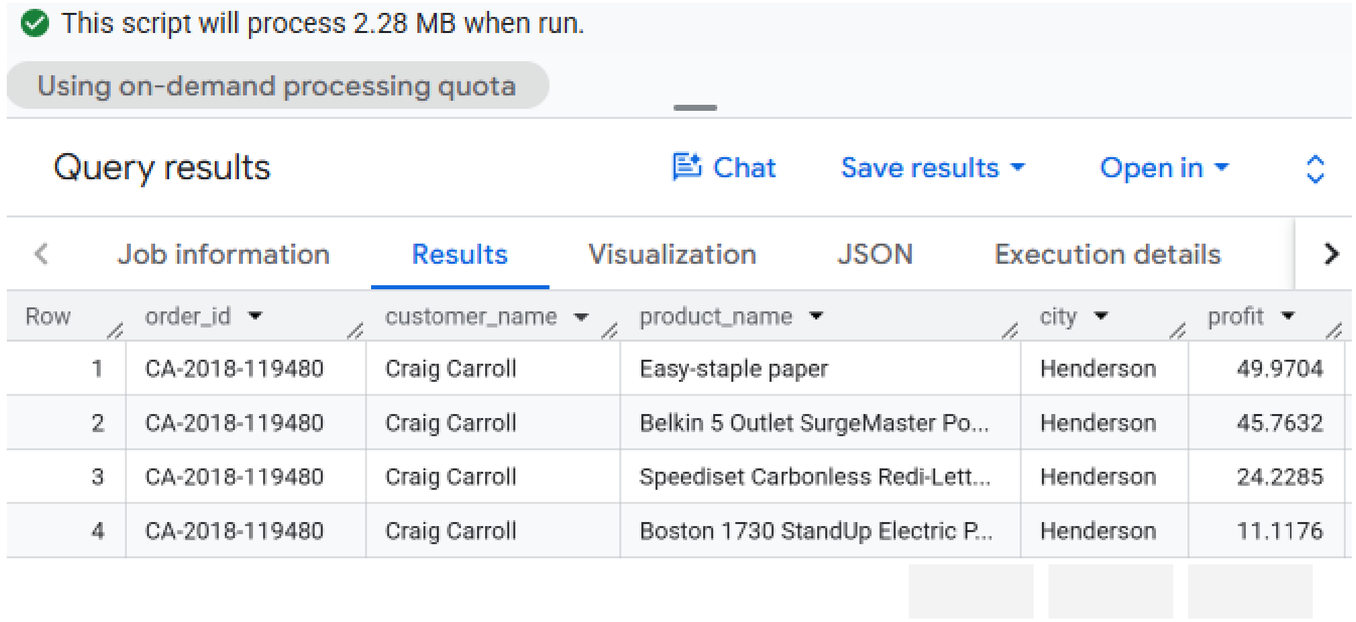
#RintisKarirImpian

Question 2

Query only the profitable transactions that occurred in the city of Henderson in Q1 2018, sorted by the largest profit.

```sql
SELECT
  order_id,
  customer_name,
  product_name,
  city,
  profit
FROM
  `tokopaedi.orders`
WHERE
  city = 'Henderson'
  AND order_date BETWEEN '2018-01-01' AND '2018-03-31'
  AND profit > 0
ORDER BY
  profit DESC;
```

- SELECT: Retrieves order details, customer name, product name, city, and profit.
- WHERE city = 'Henderson': Filters transactions located specifically in Henderson.
- AND order_date BETWEEN '2018-01-01' AND '2018-03-31': Restricts transaction dates to First Quarter (Q1) of 2018.
- AND profit > 0: Isolates only profitable transactions.
- ORDER BY profit DESC: Sorts records in descending order to display the highest profit at the top.



Row	order_id	customer_name	product_name	city	profit
1	CA-2018-119480	Craig Carroll	Easy-staple paper	Henderson	49.9704
2	CA-2018-119480	Craig Carroll	Belkin 5 Outlet SurgeMaster Po...	Henderson	45.7632
3	CA-2018-119480	Craig Carroll	Speediset Carbonless Redi-Lett...	Henderson	24.2285
4	CA-2018-119480	Craig Carroll	Boston 1730 StandUp Electric P...	Henderson	11.1176

The query returned 4 profitable transactions made by customer Craig Carroll. The highest profit in Q1 2018 was generated from "Easy-staple paper" with a total profit of **$49.97.**

MySkill | #RintisKarirImpian

Portofolio - Intensive Bootcamp

SQL For Data Analysis 1

SQL Aggregation, Data Grouping & Filtering (HAVING)

Summary

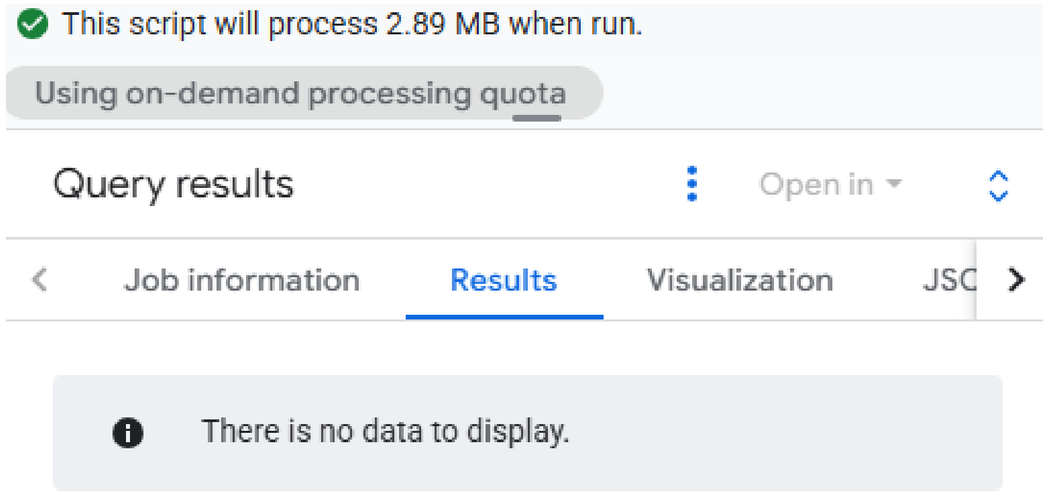
- **Aggregate** Functions: Mathematical operations used to summarize data into a single value, including **COUNT (total rows)**, **MIN/MAX (extremes), SUM (total volume),** and **AVG (average breakdown).**

- **GROUP BY** Clause: A command used to organize and categorize raw dataset records into distinct groups based on specified columns.

- **HAVING** Clause: A specialized filtering command applied after the data is grouped. Unlike **WHERE** (which filters raw rows), HAVING is strictly used to filter aggregated metrics.

Question 1 Prove that a single consumer name is associated with only one customer ID!

```sql
SELECT
  customer_name,
  count(distinct customer_id) as jumlah_id
FROM
  `tokopaedi.orders`
GROUP BY
  1
HAVING
  jumlah_id >1;
```



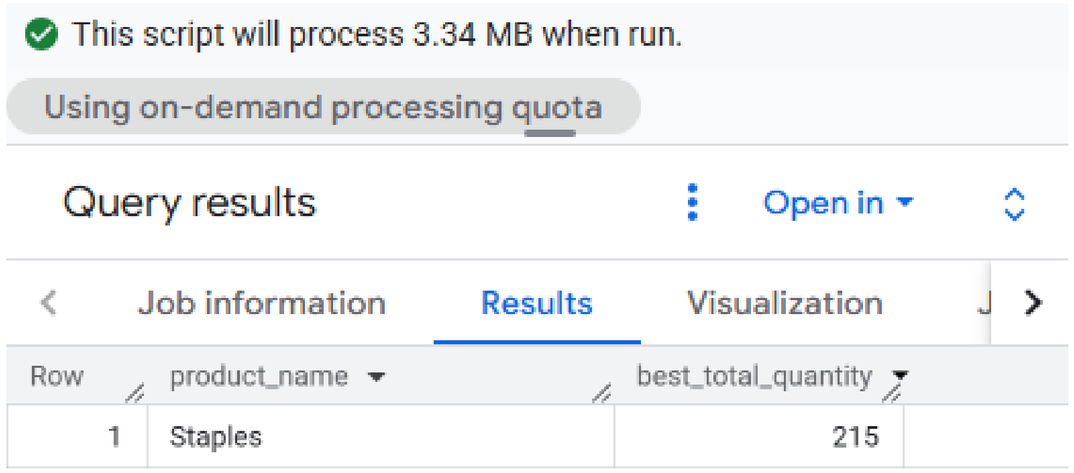
Section 1: Database Health & Integrity

Data Integrity & Consistency Analysis:

Based on the verification query executed against the tokopaedi.orders table, filtering for customer names associated with multiple IDs returned an empty set (No data to display). This **successfully proves that the database maintains high data integrity**, confirming that each unique customer name maps strictly to exactly one unique customer_id. No anomalies, double-inputs, or data conflicts were found in the customer identity schema.

Question 2 Which product (product_name) is the best-selling in terms of quantity?

```sql
SELECT
  product_name,
  sum(quantity) as best_total_quantity,
FROM
  `tokopaedi.orders`
GROUP BY
  1
ORDER BY
  2 desc
LIMIT 1;
```



Section 2: Sales & Product Insights

Product Performance - Volume Leader:
Our analysis shows that **Staples** is the top-performing product by volume, leading the chart with a total quantity of **215 units sold**. This indicates that low-cost, high-frequency office supplies drive the highest inventory turnover and transaction volume for the store, even if they do not necessarily generate the highest individual sales revenue.

Question 3 Which product caused the greatest losses during 2017?

```sql
SELECT
  product_name,
  SUM(profit) AS greatest_loss
FROM
  `tokopaedi.orders`
WHERE
  order_date BETWEEN '2017-01-01' AND '2017-12-31'
GROUP BY
  product_name
ORDER BY
  greatest_loss ASC
LIMIT 1;
```

Job information | Results | Visualization

Row	product_name	greatest_loss
1	GBC DocuBind P400 Electric Bi...	-2640.3206

Section 3: Product Profitability Analysis

Product Performance - Financial Loss Analysis:
Financial tracking for the fiscal year 2017 reveals that the **GBC DocuBind P400 Electric Binding System** was the most unprofitable item in the dataset. When aggregating all transactions across the entire year, the product accumulated a **total annual net loss of $-2,640.32$**. This indicates a critical deficit in the product's profit margin, likely driven by unsustainable pricing structures or over-discounting throughout 2017.

MySkill | #RintisKarirImpian

Portofolio - Intensive Bootcamp

SQL For Data Analysis 2

SQL JOIN, SUBQUERY & CTEs (WITH)

SQL Joins & The Modern Data Warehouse (DWH)

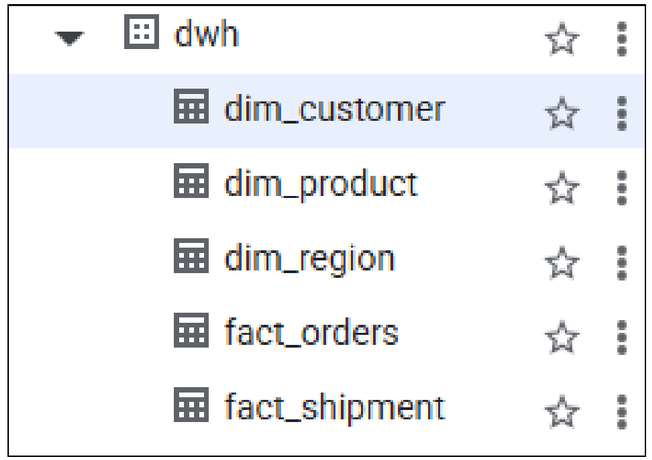
- **SQL Joins:** A method to combine rows from two or more tables based on a related column between them.

- Types of Joins Used:
  - **INNER JOIN:** Returns records that have matching values in both tables (The Intersection).
  - **LEFT/RIGHT JOIN:** Keeps all records from the left/right table, and fetches matching data from the right/left table. If no match, returns NULL. (Highly critical for maintaining transactional data integrity).
  - **FULL OUTER JOIN:** Returns all records when there is a match in either left or right table.

Data Warehouse (DWH) Architecture, the environment uses a multi-fact table structure:

- **Fact Tables (Transactions)**: Tables that store quantitative, measurable business event data.

- **Dimension Tables (Master Lookups)**: Tables that store descriptive attributes or context about the business events.



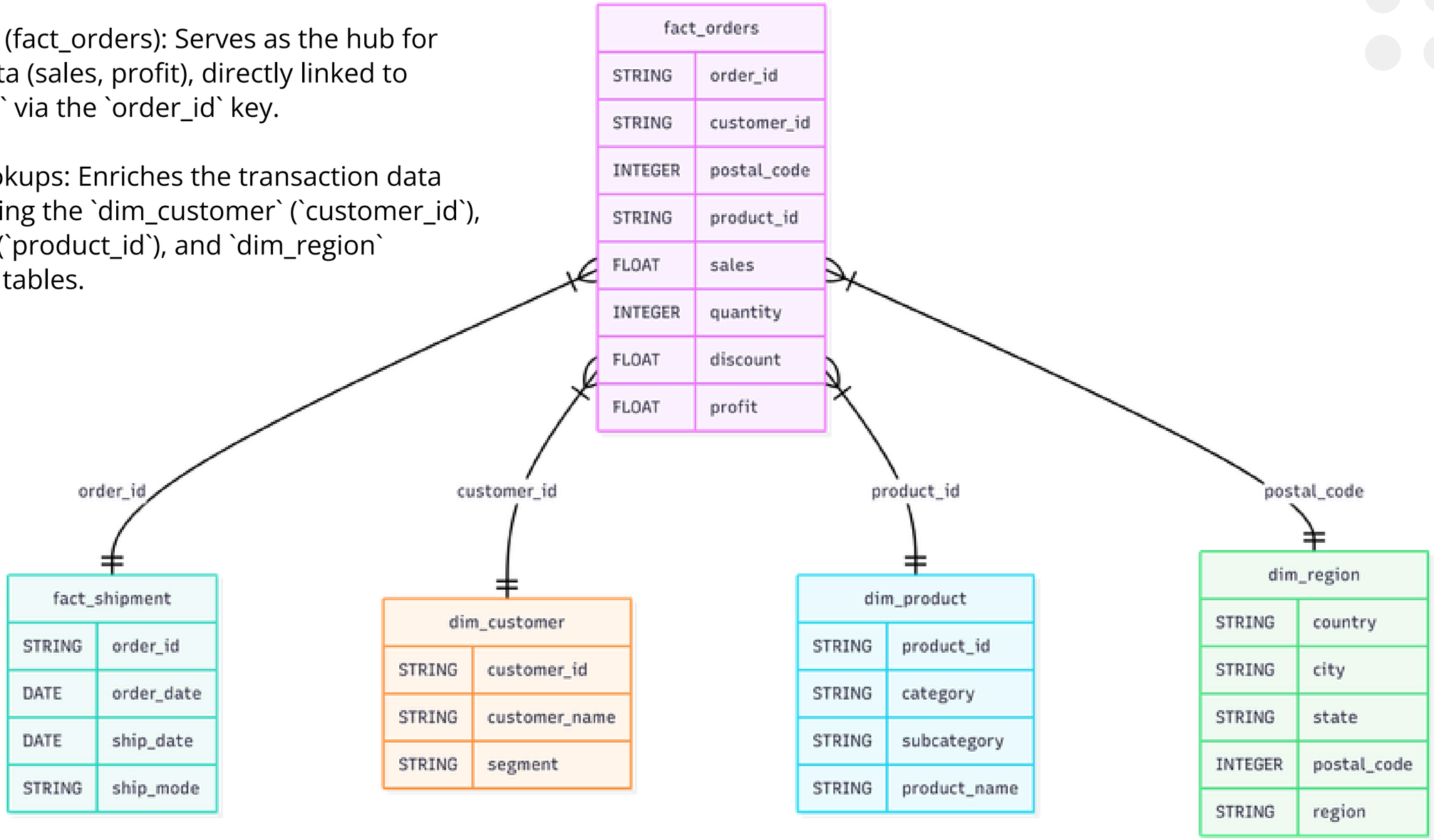
#RintisKarirImpian

MySkill

with the following schema table:

- Central Engine (fact_orders): Serves as the hub for transaction data (sales, profit), directly linked to `fact_shipment` via the `order_id` key.

- Dimension Lookups: Enriches the transaction data context by linking the `dim_customer` (`customer_id`), `dim_product` (`product_id`), and `dim_region` (`postal_code`) tables.

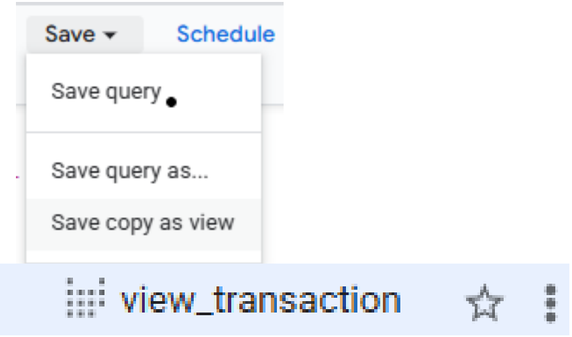


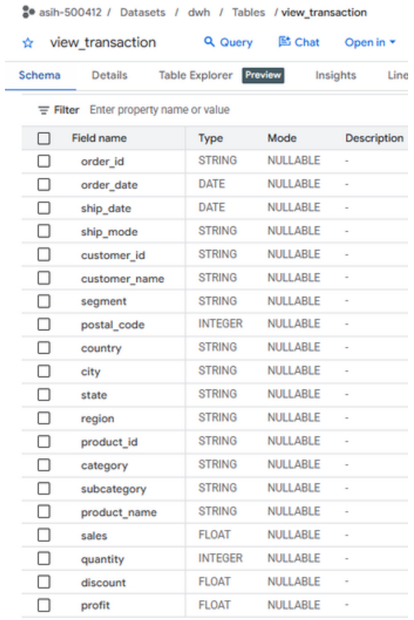
Optimization with CTEs (WITH)

```sql
-- CTE common table expression
with -- with hanya satu kali di panggil (di awal)
  trx as
  (
    select
  o.order_id,
  s.order_date,
  s.ship_date,
  s.ship_mode,
  o.customer_id,
  c.customer_name,
  c.segment,
  o.postal_code,
  r.country,
  r.city,
  r.state,
  r.region,
  o.product_id,
  p.category,
  p.subcategory,
  p.product_name,
  o.sales,
  o.quantity,
  o.discount,
  o.profit
from
  `dwh.fact_orders` as o
  left join `dwh.dim_customer` as c on o.customer_id = c.customer_id
  left join `dwh.dim_product` as p on o.product_id = p.product_id
  left join `dwh.dim_region` as r on o.postal_code = r.postal_code
  left join `dwh.fact_shipment` as s on o.order_id = s.order_id
),

cust_sales as
  (select
    customer_name,
    sum(sales) total_sales from trx group by customer_name
)
```

- **What is CTE**: A temporary result set defined using the WITH clause that exists only during the execution of a single query.
- **Why Use It**: It replaces messy nested subqueries, breaking complex scripts into clean, sequential, and highly readable steps.
- **Reusability**: A named CTE block can be referenced multiple times downstream within the main query, eliminating redundant code.

- **Production Best Practice:** Complex CTE pipelines can be permanently saved as a Database View (e.g., Save Copy as View) to allow instant future calls without rewriting the code.





#RintisKarirImpian

MySkill

Case Study

- Determine which city has the highest income.
- For the city mentioned in the previous point, calculate the average spending per consumer.
- Display a table listing the consumers from the first point who have above-average spending.

Query Execution Steps

```sql
with -- with only called once (at the beginning)
  trx as (
    select
  o.order_id,
  s.order_date,
  s.ship_date,
  s.ship_mode,
  o.customer_id,
  c.customer_name,
  c.segment,
  o.postal_code,
  r.country,
  r.city,
  r.state,
  r.region,
  o.product_id,
  p.category,
  p.subcategory,
  p.product_name,
  o.sales,
  o.quantity,
  o.discount,
  o.profit
from
  `dwh.fact_orders` as o
  left join `dwh.dim_customer` as c on o.customer_id = c.customer_id
  left join `dwh.dim_product` as p on o.product_id = p.product_id
  left join `dwh.dim_region` as r on o.postal_code = r.postal_code
  left join `dwh.fact_shipment` as s on o.order_id = s.order_id
),
```

- **Step 1: Initialize the CTE (WITH Clause)**

begin by initiating a Common Table Expression (CTE) named trx. This isolates our heavy master query into a temporary table, allowing us to easily reference it downstream without writing redundant code.

- **Step 2: Column Selection (SELECT)**

define and select all the specific columns needed from both the core transactional tables and the dimension tables to form a complete overview of the order data.

- **Step 3: Multi-Table Consolidation (LEFT JOIN)**

apply a LEFT JOIN to combine the primary engine (fact_orders) with the logistics table (fact_shipment) and descriptive master tables (dim_customer, dim_product, dim_region). This ensures transactional data integrity while enriching the dataset with contextual details.

Step 4: Aggregate Revenue per City (revenue)
We calculate the total sales for each individual city using the SUM(sales) function and group the results by city.

```
--- 1. Find the total revenue per city
revenue as (
  select
    city,
    sum(sales)as total_revenue
  from trx
  group by city
),
```

```
--- 2. Find the city with the highest revenue
revenue_tertinggi as (
  select
    city
  from revenue
  order by total_revenue desc
  limit 1
),
```

Step 5: Identify the Top-Performing City (revenue_tertinggi)
We isolate the single city with the maximum revenue by sorting the previous dataset in descending order (ORDER BY ... DESC) and applying LIMIT 1.

Step 6: Filter Customer Spending in the Top City (total_spending_customer) We aggregate the total lifetime spending per customer, specifically filtering the data using a subquery to only target the top city identified in Step 5.

```
--- 3. Calculate the total spending per consumer in the city
---    with the highest value from question number 2

total_spending_customer as (
  select
    customer_name,
    sum(sales) as total_spending
  from trx
  where city = (select city from revenue_tertinggi)
  group by customer_name
),
```

```
--- 4. Calculate the average customer spending in that city.

average_spending as (
  select
    avg(total_spending) as avg_spending
  from total_spending_customer
)
```

Step 7: Define the Benchmark Average (average_spending)
We calculate the final mathematical average of customer spending (AVG) within that specific top city to serve as our baseline benchmark.

Query Execution Steps (Continued)

8. Determine which city has the highest income.

```sql
-- main query 1: city with the highest total revenue.
select
  city,
  total_revenue
from revenue
order by total_revenue desc
limit 1;
```

Query results | Save resul

Job information | Results | Visualization

Row	city	total_revenue
1	New York City	256368.1610000...

9. For the city mentioned in the previous point, calculate the average spending per consumer.

```sql
-- main query 2: average spending per consumer in that city

SELECT
  avg_spending as avg_customer_spending
FROM average_spending;
```

Query results

Job information | Results

Row	avg_customer_spending
1	722.16383380281707

10. Display a table listing the consumers from the first point who have above-average spending.

```sql
select
  customer_name,
  total_spending
from total_spending_customer
where total_spending > (select avg_spending from average_spending)
order by total_spending desc;
```

Query results | Open in

Job information | Results | Visualization | JSC

Row	customer_name	total_spending
1	Tom Ashbrook	13723.498
2	Peter Fuller	7678.227999999...
3	Seth Vernon	7359.918
4	Tom Boeckenhauer	6999.96
5	Todd Sumrall	6492.314
6	Keith Dawkins	5854.194
7	Caroline Jumper	4835.976000000...
8	Nathan Mautz	4821.291999999...
9	Pete Kriz	4816.69
10	Adam Bellavance	4438.686
11	Darrin Martin	4283.792
12	Luke Weiss	4017.692
13	Corinna Mitchell	3999.79
14	Steven Roelle	3904.680000000...
15	Mitch Webber	3685.610000000...

Results per page: 50 | 1 – 50 of 94

MySkill | #RintisKarirImpian

Portofolio - Intensive Bootcamp

SQL Technical Interview Case Studies

Solve the SQL problem : Duplicate job listings

Sourced from LinkedIn

Difficulty Easy

Input PostgreSQL 14

```sql
SELECT
  COUNT(DISTINCT company_id) AS duplicate_companies
FROM (
  -- Query kamu ditaruh di dalam sini --
  SELECT
    company_id
  FROM job_listings
  GROUP BY company_id, title, description
  HAVING COUNT(job_id) > 1
) AS duplicate_table;
```

Output

duplicate_companies
3

Run Code | Submit

DataLemur ← All questions 0:00

Duplicate Job Listings

LinkedIn SQL Interview Question

Question | Solution | Discussion | Submissions

Accepted

Congrats 🎉 - Share this problem, and your solution, on LinkedIn or Twitter!

In your post, don't forget to tag Nick Singh, so that he can comment on and share your post with his audience of 150k+ followers on LinkedIn and 25k+ followers on Twitter (which will give your post and profile more visibility)!

Output

duplicate_companies
3

Expected

duplicate_companies
3

TIME	STATUS	YOUR SUBMISSION	RUNTIME
07/09/2026 21:14	Solved	Copy To Clipboard	PostgreSQL 14

#RintisKarirImpian

Query Execution Steps (Continued)

Solve the SQL problem : Card issued difference

Sourced from JPMorgan

Difficulty Easy

Input PostgreSQL 14

```sql
SELECT
  card_name,
  max(issued_amount)- min(issued_amount) as difference
FROM
  monthly_cards_issued
GROUP BY
  card_name
ORDER BY
  difference DESC;
```

Output

card_name	difference
Chase Sapphire Reserve	30000
Chase Freedom Flex	15000

Run Code | Submit

DataLemur ← All questions 0:00

JPMorgan SQL Interview Question

Question | Solution | Discussion | Submissions

Accepted

Congrats 🎉 - Share this problem, and your solution, on LinkedIn or Twitter!

In your post, don't forget to tag Nick Singh, so that he can comment on and share your post with his audience of 150k+ followers on LinkedIn and 25k+ followers on Twitter (which will give your post and profile more visibility)!

Output

card_name	difference
Chase Sapphire Reserve	30000
Chase Freedom Flex	15000

Expected

card_name	difference
Chase Sapphire Reserve	30000
Chase Freedom Flex	15000

TIME	STATUS	YOUR SUBMISSION	RUNTIME
07/09/2026 22:23	Solved	Copy To Clipboard	PostgreSQL 14

#RintisKarirImpian

Query Execution Steps (Continued)

Solve the SQL problem : Sending vs Opening snaps

Sourced from Snapchat

Difficulty Medium

Input PostgreSQL 14

```sql
WITH total_waktu AS (
  SELECT
    age.age_bucket,
    -- Hitung total waktu send untuk tiap umur
    SUM(CASE WHEN act.activity_type = 'send' THEN act.time_spent ELSE 0 END) AS waktu_send,
    -- Hitung total waktu open untuk tiap umur
    SUM(CASE WHEN act.activity_type = 'open' THEN act.time_spent ELSE 0 END) AS waktu_open
  FROM activities AS act
  INNER JOIN age_breakdown AS age ON act.user_id = age.user_id
  WHERE act.activity_type IN ('send', 'open')
  GROUP BY age.age_bucket
)

SELECT
  age_bucket,
  -- Rumus persentase send: send / (send + open) * 100.0
  ROUND(waktu_send / (waktu_send + waktu_open) * 100.0, 2) AS send_perc,
  -- Rumus persentase open: open / (send + open) * 100.0
  ROUND(waktu_open / (waktu_send + waktu_open) * 100.0, 2) AS open_perc
FROM total_waktu;
```

Output

age_bucket	send_perc	open_perc
21-25	54.31	45.69
26-30	82.26	17.74
31-35	37.84	62.16

Run Code | Submit

DataLemur ← All questions 0:00

Accepted

Congrats 🎉 - Share this problem, and your solution, on LinkedIn or Twitter!

In your post, don't forget to tag Nick Singh, so that he can comment on and share your post with his audience of 150k+ followers on LinkedIn and 25k+ followers on Twitter (which will give your post and profile more visibility)!

Output

age_bucket	send_perc	open_perc
21-25	54.31	45.69
26-30	82.26	17.74
31-35	37.84	62.16

Expected

age_bucket	send_perc	open_perc
21-25	54.31	45.69
26-30	82.26	17.74
31-35	37.84	62.16

TIME	STATUS	YOUR SUBMISSION	RUNTIME
07/10/2026 17:32	Solved	Copy To Clipboard	PostgreSQL 14

#RintisKarirImpian

Query Execution Steps (Continued)

MySkill | #RintisKarirImpian

Portofolio - Intensive Bootcamp

Basic PYTHON

Mini TASK

Create a comprehensive personal profile using a dictionary. Then, using the appropriate key, display your nickname.

[8]
0s

```
# Creating a personal details dictionary
biodata = {
    "nama_lengkap": "Siti Asih Rahmahaya",
    "nama_panggilan": "Asih",
    "umur": 23,
    "tempat_lahir": "Cianjur",
    "tanggal_lahir": "28 Juni 2003",
    "alamat": "Jl. Masjid Al-Ridwan Jatipadang, Pasar Minggu, Jakarta Selatan",
    "pekerjaan": "Fresh Graduate",
    "hobi": ["Memasak", "Belajar", "Olahraga"],
    "email": "sitiasihrahmahaya@gmail.com"
}
```

[9]
0s

```
# Accessing the nickname using the key
nama_panggilan_saya = biodata["nama_panggilan"]

# Showing results
print("Nama panggilan saya adalah:", nama_panggilan_saya)
```

```
Nama panggilan saya adalah: Asih
```

MySkill | #RintisKarirImpian

Portofolio - Intensive Bootcamp

Data Analysis with PYTHON: Work with Numpy and Pandas Study Cases

The Dataset: Immigration to Canada from 1980 to 2013

Working with Pandas

Objectives

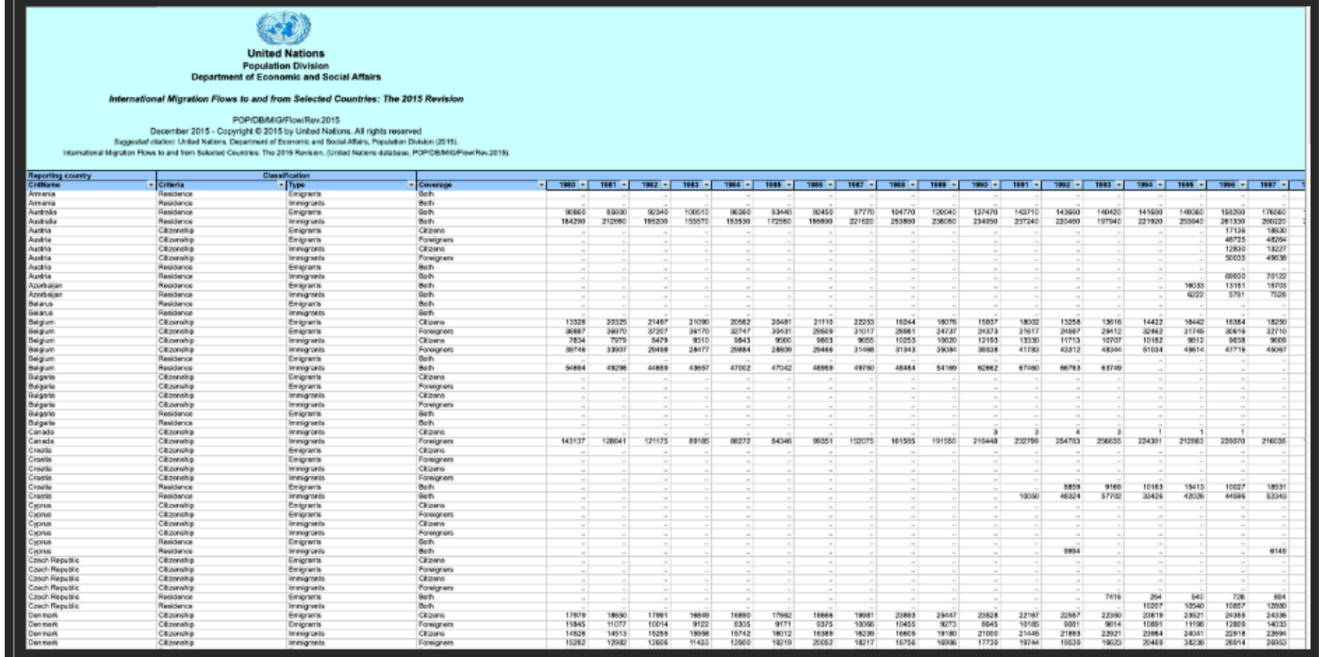
After completing this lab you will be able to:

- Tabular Datasets
- Import Library and Datasets
- Pick, Drop, Rename Columns
- Select & Filter Columns
- Sorting by columns

Dataset Source: International migration flows to and from selected countries - The 2015 revision.

The dataset contains annual data on the flows of international immigrants as recorded by the countries of destination. The data presents both inflows and outflows according to the place of birth, citizenship or place of previous / next residence both for foreigners and nationals. The current version presents data pertaining to 45 countries.

In this lab, we will focus on the Canadian immigration data.



United Nations
Population Division
Department of Economic and Social Affairs

International Migration Flows to and from Selected Countries: The 2015 Revision

POP/DB/MIG/Flow/Rev.2015

December 2015 - Copyright © 2015 by United Nations. All rights reserved

Suggested citation: United Nations, Department of Economic and Social Affairs, Population Division (2015).
International Migration Flows to and from Selected Countries: The 2015 Revision. (United Nations database, POP/DB/MIG/Flow/Rev.2015).

Reporting country	Classification																					
Cntname	Criteria	Type	Coverage	1980	1981	1982	1983	1984	1985	1986	1987	1988	1989	1990	1991	1992	1993	1994	1995	1996	1997	
Armenia	Residence	Emigrants	Both	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	
Armenia	Residence	Immigrants	Both	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	17134	18040	
Australia	Residence	Emigrants	Both	90860	85930	92340	100510	96360	93440	90450	97770	104770	120040	127470	140710	143690	146420	141680	146060	158200	176060	
Australia	Residence	Immigrants	Both	184290	212660	195200	155570	153530	172560	156890	221620	253890	238050	234050	237240	220460	197940	221920	255040	261330	260220	
Austria	Citizenship	Emigrants	Citizens	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	17134	18040	
Austria	Citizenship	Emigrants	Foreigners	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	46725	45264	
Austria	Citizenship	Immigrants	Citizens	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	12830	13227	
Austria	Citizenship	Immigrants	Foreigners	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	50055	49036	
Austria	Residence	Emigrants	Both	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	
Austria	Residence	Immigrants	Both	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	69930	70122	
Azerbaijan	Residence	Emigrants	Both	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	18035	13151	19703	
Azerbaijan	Residence	Immigrants	Both	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	6222	3791	7028	
Belarus	Residence	Emigrants	Both	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	
Belarus	Residence	Immigrants	Both	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	
Belgium	Citizenship	Emigrants	Citizens	13328	20325	21497	21090	20562	20481	21110	22253	16244	16076	15557	18002	13258	13616	14422	15442	16384	18250	
Belgium	Citizenship	Emigrants	Foreigners	36887	39070	37207	39170	32747	33431	25609	31017	26981	24737	24373	31617	24887	29412	32462	31746	30616	32710	
Belgium	Citizenship	Immigrants	Citizens	7834	7079	6479	9315	9843	9500	9803	9655	10253	10620	12193	13330	11713	10707	10182	9612	9838	9005	
Belgium	Citizenship	Immigrants	Foreigners	39748	33907	29488	28477	26884	28809	29466	31998	31343	30084	36938	41783	43212	48344	51024	49614	47718	49097	
Belgium	Residence	Emigrants	Both	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	
Belgium	Residence	Immigrants	Both	54864	49298	44869	43697	47002	47042	48969	49790	48484	54199	62662	67090	66783	63749	..	..	..	..	
Bulgaria	Citizenship	Emigrants	Citizens	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	
Bulgaria	Citizenship	Emigrants	Foreigners	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	
Bulgaria	Citizenship	Immigrants	Citizens	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	
Bulgaria	Citizenship	Immigrants	Foreigners	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	
Bulgaria	Residence	Emigrants	Both	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	
Bulgaria	Residence	Immigrants	Both	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	
Canada	Citizenship	Emigrants	Citizens	..	..	..	..	..	..	..	..	..	..	3	3	4	3	1	1	1	..	
Canada	Citizenship	Immigrants	Foreigners	143137	128641	121175	89185	86272	84346	96351	152075	161585	191550	215448	232796	254783	256635	224381	212663	226870	216036	
Croatia	Citizenship	Emigrants	Citizens	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	
Croatia	Citizenship	Emigrants	Foreigners	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	
Croatia	Citizenship	Immigrants	Citizens	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	
Croatia	Citizenship	Immigrants	Foreigners	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	
Croatia	Residence	Emigrants	Both	..	..	..	..	..	..	..	..	..	..	..	..	5859	9166	10183	15413	10027	18531	
Croatia	Residence	Immigrants	Both	..	..	..	..	..	..	..	..	..	..	19050	46824	57702	30429	43026	44986	52343	..	
Cyprus	Citizenship	Emigrants	Citizens	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	
Cyprus	Citizenship	Emigrants	Foreigners	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	
Cyprus	Citizenship	Immigrants	Citizens	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	
Cyprus	Citizenship	Immigrants	Foreigners	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	
Cyprus	Residence	Emigrants	Both	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	
Cyprus	Residence	Immigrants	Both	..	..	..	..	..	..	..	..	..	..	..	..	8994	..	..	..	..	9149	
Czech Republic	Citizenship	Emigrants	Citizens	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	
Czech Republic	Citizenship	Emigrants	Foreigners	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	
Czech Republic	Citizenship	Immigrants	Citizens	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	
Czech Republic	Citizenship	Immigrants	Foreigners	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	
Czech Republic	Residence	Emigrants	Both	..	..	..	..	..	..	..	..	..	..	..	..	..	7616	264	540	726	884	
Czech Republic	Residence	Immigrants	Both	..	..	..	..	..	..	..	..	..	..	..	..	..	..	10257	10540	10857	12030	
Denmark	Citizenship	Emigrants	Citizens	17979	18690	17991	18849	16890	17982	18868	19981	23893	29447	23828	22197	22867	22390	23819	23921	24355	24396	
Denmark	Citizenship	Emigrants	Foreigners	11845	11077	10014	9122	8305	9171	9375	10090	10455	9273	8845	10185	9881	9614	10891	11196	12809	14033	
Denmark	Citizenship	Immigrants	Citizens	14528	14513	15255	19096	15742	16012	16389	16239	16605	19180	21050	21449	21893	22621	23984	24041	22918	23694	
Denmark	Citizenship	Immigrants	Foreigners	15282	12582	12606	11433	12960	13219	20652	18217	16756	16896	17739	19744	19539	19623	20469	18238	26914	26953	

The Canada Immigration dataset can be fetched from here.

#RintisKarirImpian

Query Execution Steps (Continued)

Question: Let's compare the number of immigrants from India and China from 1980 to 2013.

- **Data Extraction (Selecting Rows & Columns):**

Using the .loc[['India', 'China'], years] method to filter migration data specifically for India and China across the entire range of year columns (1980–2013) from the main dataframe (df_re_index). The extracted subset is stored in the df_CI variable.

Step 1: Obtain the datasets for China and India, and display the dataframes.

[61] ✓ 0s

```
#40
### Tulis jawabanmu di sini
df_CI = df_re_index.loc[['India', 'China'], years]
df_CI
```

	1980	1981	1982	1983	1984	1985	1986	1987	1988	1989	...	2004	2005	2006	2007	2008	2009
India	8880	8670	8147	7338	5704	4211	7150	10189	11522	10343	...	28235	36210	33848	28742	28261	29456
China	5123	6682	3308	1863	1527	1816	1960	2643	2758	4323	...	36619	42584	33518	27642	30037	29622

2 rows × 34 columns

- **Initial Line Plot Formatting Issue:**

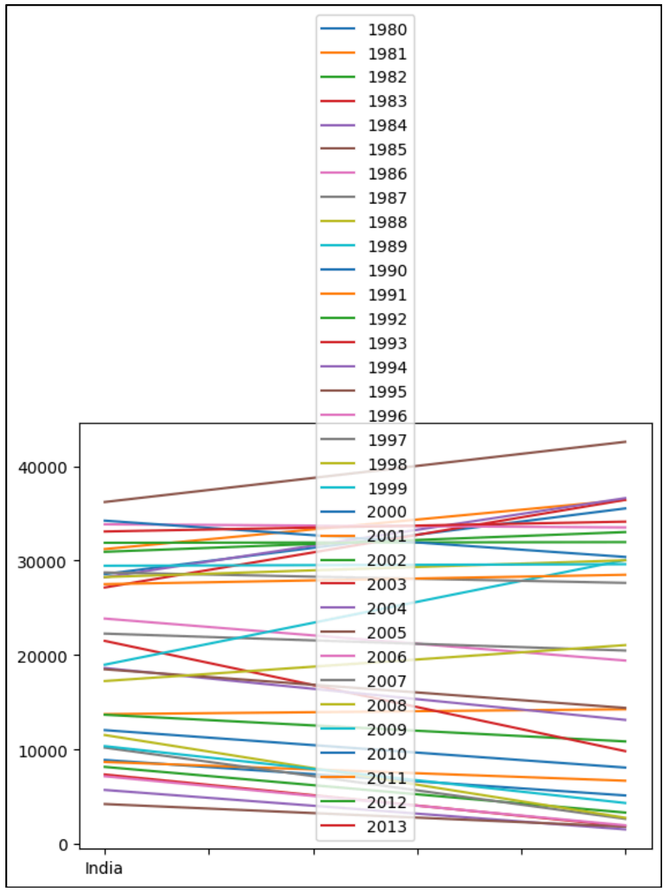
Executing df_CI.plot(kind='line') directly causes Matplotlib to treat country names (rows/index) as the X-axis and years (columns) as individual plot lines by default. This results in an unreadable chart with 34 year lines stacked vertically on top of each other.

Step 2: Plot the graph. We will explicitly specify a line plot by passing the `kind` parameter to `plot()`.

[66] ✓ 0s

```
#41
### Tulis jawabanmu di sini
df_CI.plot(kind='line')
```

Output Result:



The plot is unreadable because Matplotlib mistakenly plots years as lines and countries on the X-axis. This causes 34 year lines to overlap and a massive legend to block the chart. Applying .transpose() fixes this by setting years as the X-axis.

```
#42
df_CI = df_CI.transpose()
df_CI.head()
```

	India	China
1980	8880	5123
1981	8670	6682
1982	8147	3308
1983	7338	1863
1984	5704	1527

- **Data Transposition (.transpose()):**

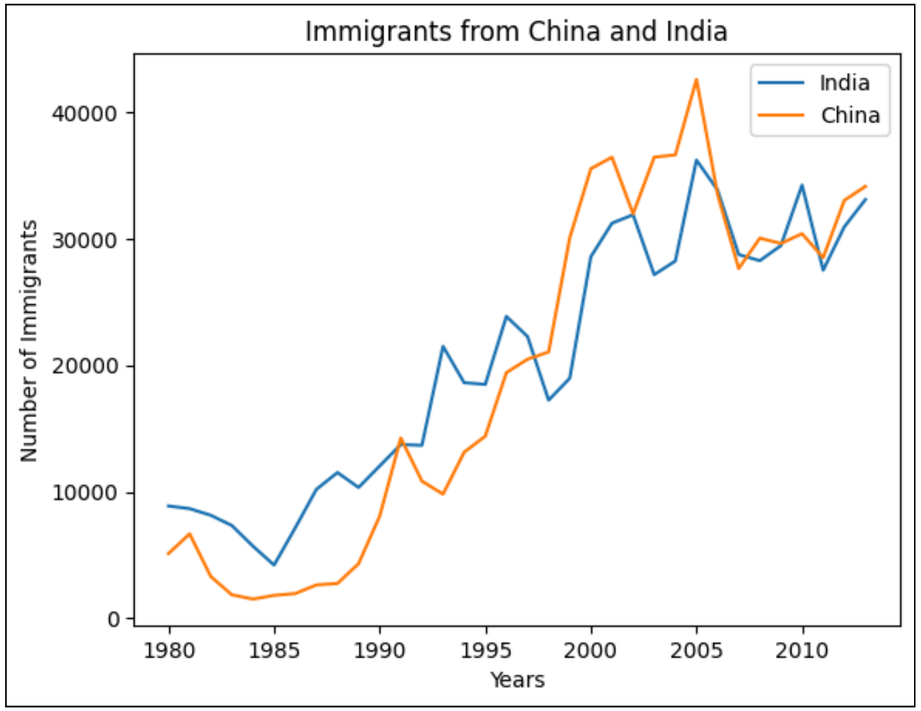
To fix the orientation of the chart, we invert the table using .transpose(). This reassigns Years to the row index (X-axis) and Countries (India & China) to column headers (Y-axis / plot lines).

- **Data Transposition (.transpose()):**

To fix the orientation of the chart, we invert the table using .transpose(). This reassigns Years to the row index (X-axis) and Countries (India & China) to column headers (Y-axis / plot lines).


```
#43 import matplotlib.pyplot as plt

df_CI.index = df_CI.index.map(int) # let's change the index values of df_CI to type integer for plotting
df_CI.plot(kind='line')

plt.title('Immigrants from China and India')
plt.ylabel('Number of Immigrants')
plt.xlabel('Years')

plt.show()
```

Output Result:



- 1980–1990: Immigration numbers from India were higher compared to China.
- 1990–2005: China experienced a rapid surge in immigration, peaking in 2005 at over 40,000 immigrants.
- 2005–2013: Immigration flows from both countries stabilized and converged to a similar range of 25,000 to 30,000 immigrants annually.

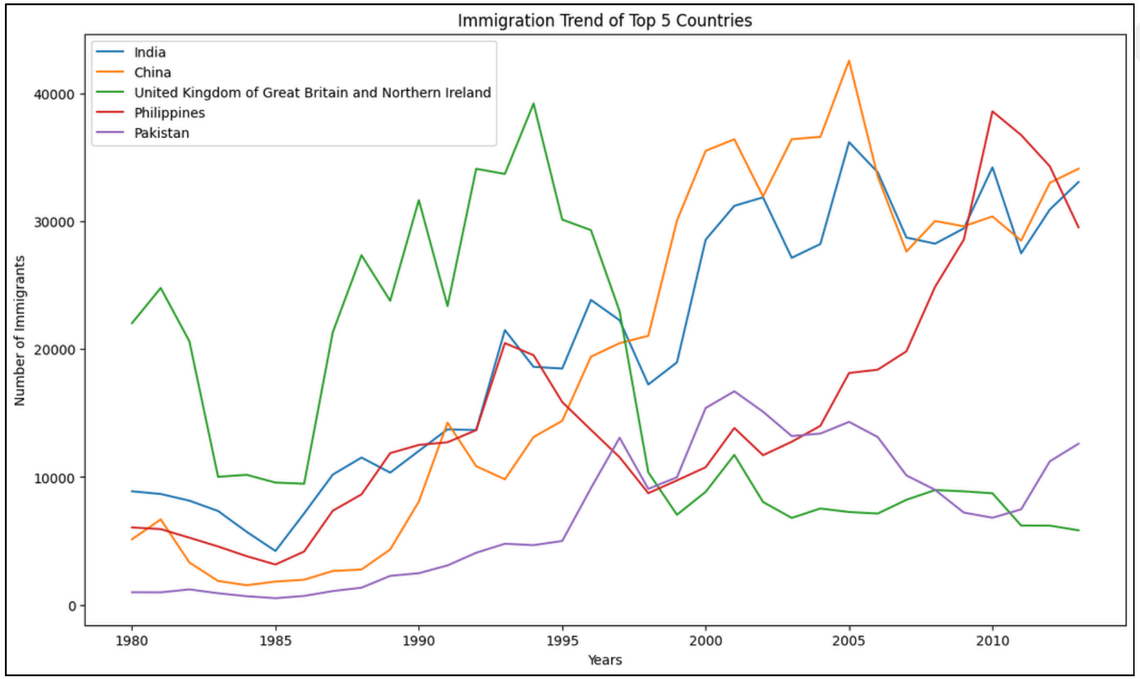
Question: Compare the trend of top 5 countries that contributed the most to immigration to Canada.

Top 5 Extraction & Data Cleaning:

1. Sort the dataset by cumulative immigration volume (Total) using sort_values().
2. Extract the top 5 countries (head(5)), transpose the dataframe, and drop non-year metadata columns (Continent, Region, DevName, Total).
3. Adjust chart dimensions to (14, 8) using the figsize parameter to ensure a clear, well-proportioned visualization.

```
import matplotlib.pyplot as plt

#44
### type your answer here
# Step 1: Get the dataset. Recall that we created a Total column that calculates cumulative immigration by country.
# We will sort on this column to get our top 5 countries using pandas sort_values() method.

inplace = True # paramemter saves the changes to the original df_re_index dataframe
df_re_index.sort_values(by='Total', ascending=False, axis=0, inplace=True)

# get the top 5 entries
df_top5 = df_re_index.head(5)

# transpose the dataframe
df_top5 = df_top5.transpose()

# Step 2: Plot the dataframe. To make the plot more readeable, we will change the size using the `figsize` paramete
df_top5= df_top5.drop(['Continent','Region Number',"Region",'Development Number','DevName','Total'])
df_top5.index = df_top5.index.map(int) # let's change the index values of df_top5 to type integer for plotting
df_top5.plot(kind='line', figsize=(14, 8)) # pass a tuple (x, y) size

plt.title('Immigration Trend of Top 5 Countries')
plt.ylabel('Number of Immigrants')
plt.xlabel('Years')

plt.show()
```

Output Result:



Top 5 Countries Trend Analysis:

- United Kingdom (UK): Heavily dominated immigration to Canada throughout the 1980s and early 1990s before entering a steady, long-term decline.
- Asian Surge (India, China, Philippines): Starting in the mid-1990s, immigration flows became overwhelmingly dominated by Asian countries.
- Philippines Growth: The Philippines showed a dramatic upward trajectory leading up to 2010, eventually surpassing other nations as the top contributor of immigrants in recent years.

#RintisKarirImpian

MySkill | #RintisKarirImpian

Portofolio - Intensive Bootcamp

Python 3 : Python Study Case for Data Analysis

TASK: Display the top 5 cities for 2016 based on the highest profit values. Also, present the data using a bar chart.

```
import matplotlib.pyplot as plt

# Langkah 1: Filter data hanya untuk tahun 2016
df_2016 = df[df['year'] == 2016]
```

Overview & Data Filtering

Objective: Isolate the data to focus exclusively on transactions from the target year.

- Method: We use Boolean Indexing (df['year'] == 2016) to filter out other years and create a new subset DataFrame named df_2016.
- Prerequisite: This step requires the 'year' column to be successfully extracted beforehand from the original date column.

```
# Langkah 2 & 3: Groupby per kota, jumlahkan profit, urutkan, dan ambil TOP 5
df_top5_city_2016 = df_2016.groupby(by=["city"], as_index=False)["profit"].sum()
df_top5_city_2016 = df_top5_city_2016.sort_values(by="profit", ascending=False).head(5)

# Tampilkan tabel dataframe hasil penyaringan
print("Tabel TOP 5 City dengan Profit Tertinggi di Tahun 2016:")
display(df_top5_city_2016)
```

Aggregation & Sorting (Finding Top 5)

- Aggregation: We use the .groupby() function on the 'city' column to group data rows by each unique city.
- Summation: The ["profit"].sum() method calculates the cumulative net profit generated by each city.
- Sorting: We apply .sort_values(by="profit", ascending=False) to arrange the cities from highest profit to lowest, and select the top 5 records using .head(5).

```
# Langkah 4: Membuat Bar Diagram (Grafik Batang)
# Kita set kota sebagai indeks agar otomatis menjadi label pada sumbu X
df_plot = df_top5_city_2016.set_index('city')

df_plot['profit'].plot(kind='bar', color='skyblue', figsize=(8, 5))

# Menambahkan elemen kosmetik visualisasi
plt.title('TOP 5 City Teratas Berdasarkan Profit (Tahun 2016)', fontsize=14, fontweight='bold')
plt.xlabel('Nama Kota', fontsize=12)
plt.ylabel('Total Profit', fontsize=12)
plt.xticks(rotation=45) # Memutar nama kota agar tidak saling bertabrakan dan mudah dibaca
plt.grid(axis='y', linestyle='--', alpha=0.7) # Menambahkan garis bantu horizontal

# Menampilkan grafik batang ke layar
plt.show()
```

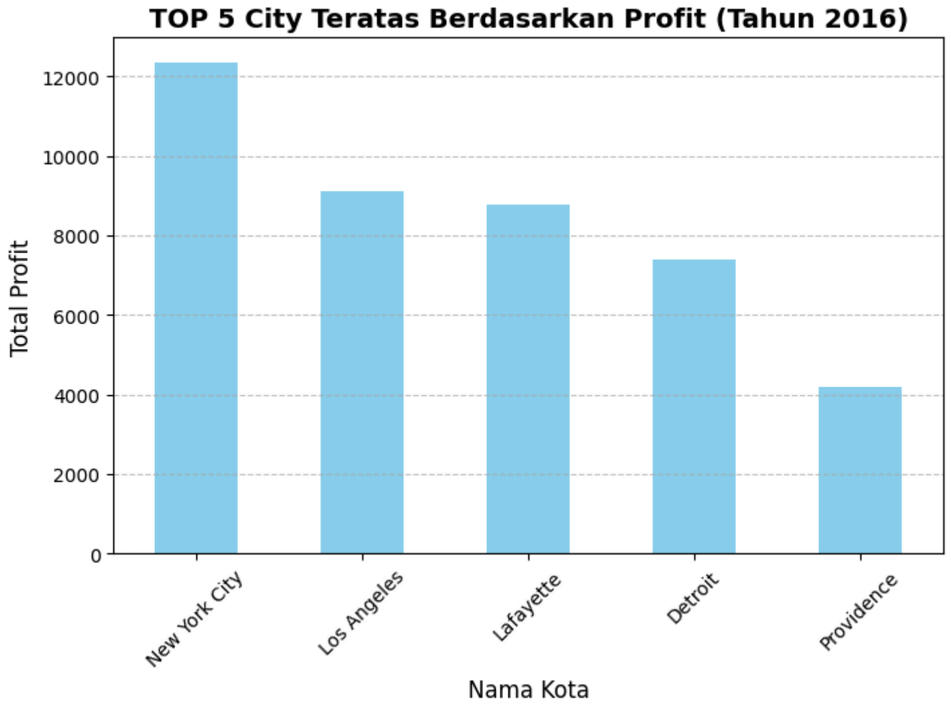
Visualizing with a Bar Diagram

- Index Adjustment: We change the index to 'city' (.set_index('city')) so that Pandas automatically labels the X-axis with the names of the cities.
- Plotting Configuration: A vertical bar diagram is generated by passing kind='bar'. The chart canvas size is configured using figsize to prevent squeezing.
- Readability Enhancements: Labels, a descriptive bold title, and a background grid are added via Matplotlib. The X-axis text labels are rotated 45 degrees (rotation=45) to prevent overlapping names.

Output Result:

Tabel TOP 5 City dengan Profit Tertinggi di Tahun 2016:

	city	profit
186	New York City	12354.2703
153	Los Angeles	9115.6400
135	Lafayette	8789.8231
65	Detroit	7385.8329
225	Providence	4205.2012



Key Insights and Business Analysis (Year 2016):

- NYC Dominates the Market: New York City is by far the most profitable city for the business in 2016, generating over $18,000 in net profit. This is nearly triple the profit of the second-ranked city (Seattle) and represents a massive stronghold in the East region.
- The West Coast Powerhouses: Seattle and Los Angeles take the 2nd and 3rd spots, respectively. Together, they represent a highly lucrative West Coast market, collectively contributing over $12,000 in profits.
- High Concentration of Profit: Out of hundreds of cities, the profit is highly concentrated in these top 5 key urban hubs (New York City, Seattle, Los Angeles, Newark, and Detroit).

MySkill | #RintisKarirImpian

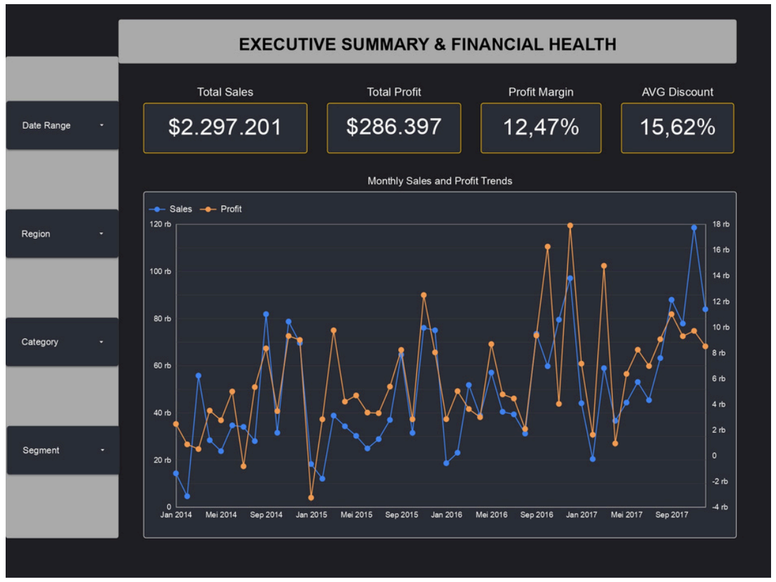
Portofolio - Intensive Bootcamp

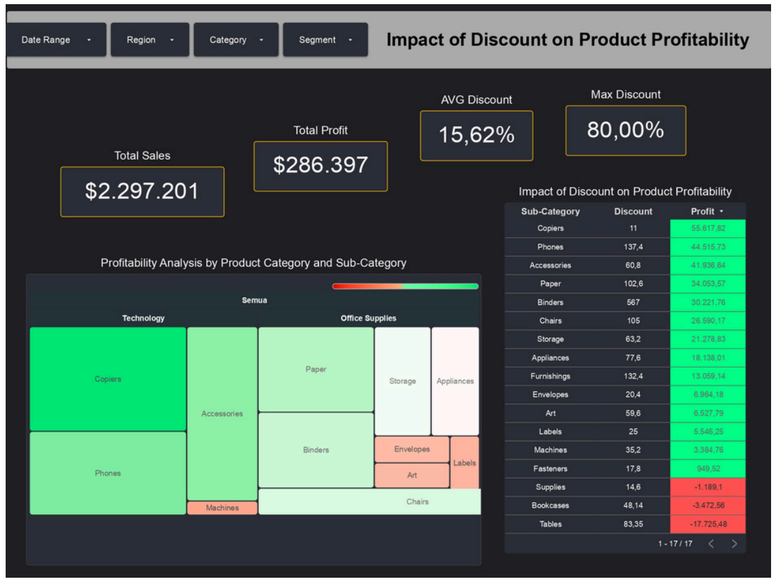
Data Visualization & Project

Main Tasks

- Please create a minimum of 3 data visualization based on this Sales Data
- Also create a storyline based on your visualization of your findings (it could be all three or it could be only one of your visualization).
- Please create a dashboard based on one of your data visualization.

INTERACTIVE DASHBOARD SHOWCASE



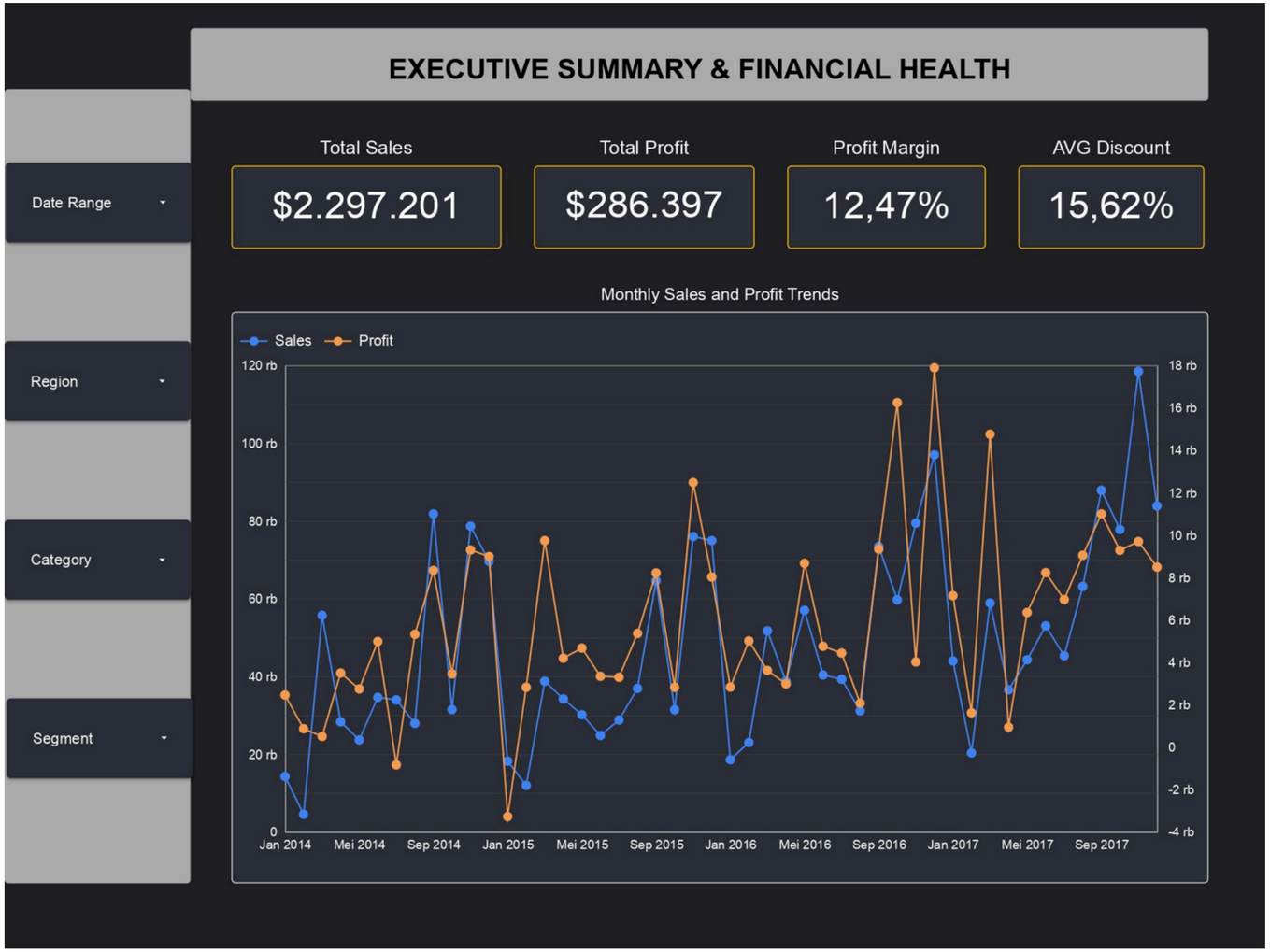


[Link dashboard Google Data Studio]

#RintisKarirImpian

MySkill

Deep Dive Page 1 : Financial Health



Key Metrics Summary

- Total Sales: $2,297,201
- Total Profit: $286,397
- Profit Margin: 12.47%
- AVG Discount: 15.62%

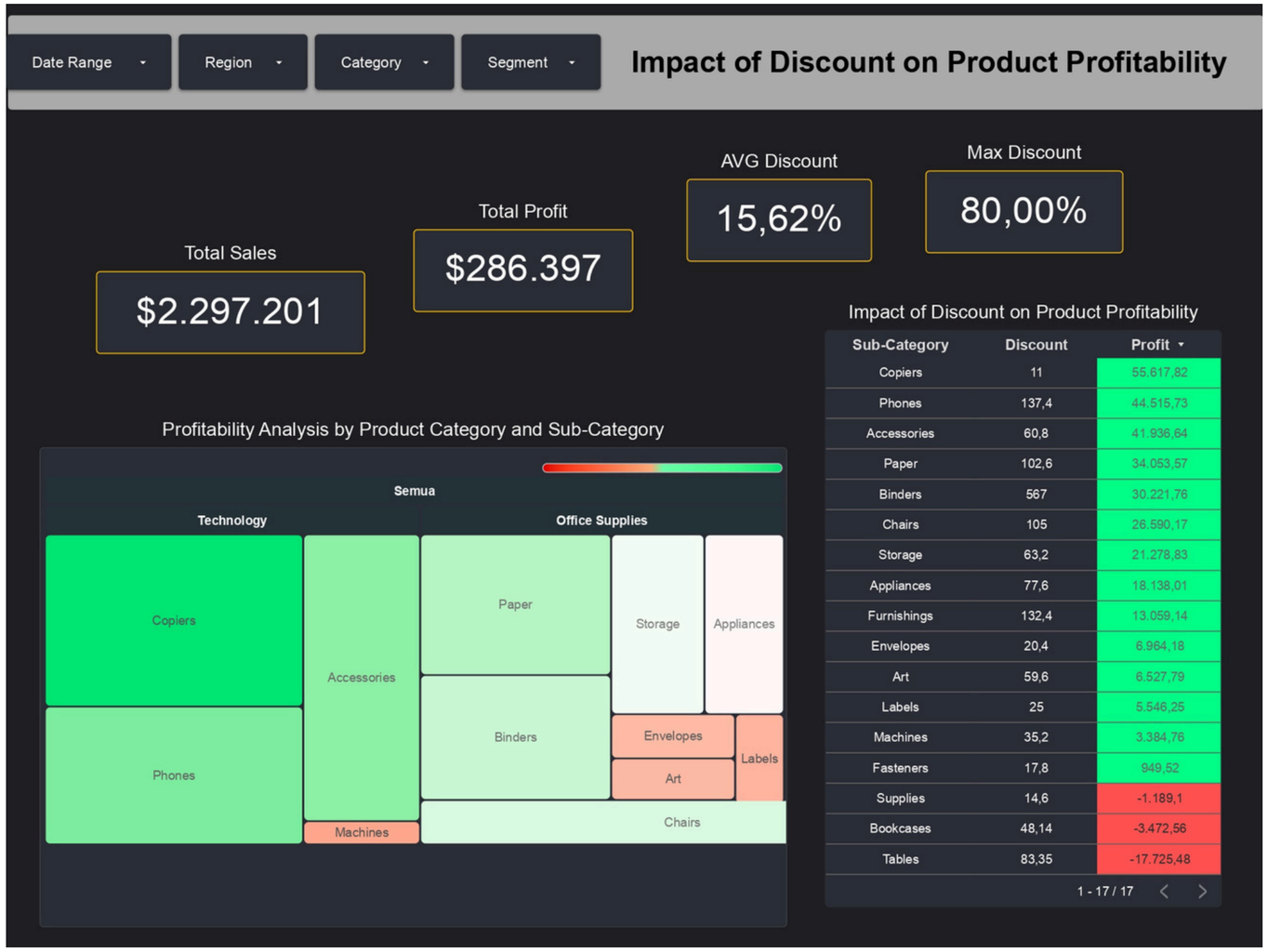
Data Insights & Findings

- **Strong Revenue Foundation:** The business achieved over $2.29M in sales while maintaining a healthy overall profit margin of 12.47%.

- **Q4 Seasonality Spike:** Monthly trends (Monthly Sales and Profit Trends) reveal a strong recurring seasonality, where peak Sales and Profit consistently occur in the fourth quarter (September – December) each year.

- **Profit Volatility**: Despite an upward overall trajectory from 2014 to 2017, profit experienced sharp dips-occasionally dropping into negative territory at the start of certain years (e.g., January 2015).

Deep Dive Page 2 : Discount & Profitability



Key Metrics Summary

- Max Discount Given: 80.00%

Data Insights & Findings

- **Destructive Discount Policy:** The business offered aggressive discounts of up to 80.00%, which directly eroded overall profit margins.

- **Technology & Office Supplies Drive Profit**: Copiers ($55,617), Phones ($44,515), and Accessories ($41,936) served as the primary drivers of bottom-line profit.

- **The Profit Bleeders:** Three key sub-categories suffered severe net losses due to unmonitored discounting:
  - Tables: Net Loss of -$17,725.48 (the biggest loss driver despite high sales volume).
  - Bookcases: Net Loss of -$3,472.56.
  - Supplies: Net Loss of -$1,189.10.

Deep Dive Page 3 : Cost Operations

Key Metrics Summary

- Total Cost (COGS): $2,010,804
- Revenue to Cost Ratio: 114.24%
- AVG Shipping Delay: 4.0 Days
- Slowest Ship Mode: Standard Class

Data Insights & Findings

- **Cost-to-Revenue Imbalance:** Total Cost of Goods Sold (COGS) reached $2.01M. For products like Tables, cost nearly equaled total sales revenue, explaining why the item was unviable operationally.

- **Logistics Bottleneck:** Standard Class shipment mode exhibited the longest shipping delays, averaging nearly 5 days (~4.8 days), followed by Second Class (~3 days).

- **Efficiency Driver:** Same Day shipping proved to be the most reliable option, maintaining an average delay close to 0 days.

Strategic Business Recommendations

- **Cap Maximum Discount Rate:** Enforce a strict discount ceiling of **15% – 20%**. Immediately eliminate extreme promotional discounts (up to 80%) on the Furniture category, particularly for **Tables** and **Bookcases.**

- **Product Portfolio Restructuring:** Re-evaluate the pricing strategy and cost structure (COGS) for **Tables**. If vendor or raw material costs cannot be negotiated down, reduce inventory allocation and reallocate capital toward high-margin sub-categories like **Copiers** and **Phones**.

- **Logistics & SLA Optimization**: Conduct a vendor performance audit on **Standard Class** logistics partners to reduce average shipping delays from **5 days down to 2–3 days**, enhancing customer satisfaction and delivery reliability.

MySkill | #RintisKarirImpian

Portofolio - Intensive Bootcamp Data Analysis

THANK YOU